МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: М. А. Самоданов
Группа: М8О-316БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы - изучение основ 3D-графики: построение простых 3D-объектов, проекция на 2D-плоскость, работа с матрицами перспективной и ортографической проекции и аффинными преобразованиями.

Выполненный вариант: №1 - куб.

Реализация выполняется на C++ с использованием Vulkan (через слой совместимости MoltenVK на macOS), GLFW и ImGui на базе предоставленного стартового кода. Программа работает в реальном времени с динамической сменой проекции и трансформаций объектов и взаимодействием через интерфейс.

Помимо базового варианта выполнены все шесть дополнительных заданий: переключение проекции, управление позицией/поворотом/растяжением, анимация по сложной траектории с управлением воспроизведением, изменение цвета фигуры, процедурные цвета вершин, отрисовка двух объектов с использованием нескольких наборов дескрипторов (VkDescriptorSet).

Метод решения

Математика. Положение каждой вершины в клип-пространстве вычисляется как произведение трёх матриц:

gl_Position = P · V · M · (x, y, z, 1)

- **M (model)** — аффинное преобразование объекта в мировых координатах, собирается каждый кадр как T(позиция + смещение по траектории) · R_x · R_y · R_z · S(масштаб). Траектория задана параметрически: r(t) = (R·cos t, 0.6R·sin 2t, R·sin t) — пространственная кривая с изменяемым радиусом R и скоростью.
- **V (view)** — матрица вида, строится функцией lookAt из положения камеры, точки цели и вектора «вверх».
- **P (proj)** — матрица проекции: glm::perspective (перспективная, угол обзора 60°, near/far = 0.1/100) либо glm::ortho (ортографическая, полувысота 1.6 с учётом aspect ratio окна). Тип проекции переключается в интерфейсе.
- Учтены особенности Vulkan: переворот оси Y клип-пространства (proj[1][1] *= -1) и диапазон глубины [0, 1] (GLM_FORCE_DEPTH_ZERO_TO_ONE).

Цвет вершины назначается процедурно по её позиции в локальной системе координат (color = pos · 0.5 + 0.5), итоговый цвет в шейдере — произведение цвета вершины на множитель, задаваемый через ColorEdit.

Объекты Vulkan. Куб (24 вершины, 36 индексов) хранится в вершинном и индексном буферах, выделенных через Vulkan Memory Allocator. Матрицы передаются в шейдеры через uniform-буферы и дескрипторные наборы: сет 0 — общие преобразования сцены (view, proj), сет 1 — личный на каждый объект (model, цвет), что позволяет рисовать несколько объектов одним пайплайном. Графический пайплайн создан со статической настройкой (вершинный формат «позиция + цвет», треугольный список, тест глубины LESS, отсечение обратных граней) и динамическим viewport/scissor, поэтому изменение размера окна не требует пересоздания пайплайна. Каждый кадр CPU обновляет uniform-буферы, записывает командный буфер (begin render pass → bind pipeline → bind descriptor sets → draw indexed) и отправляет его на исполнение; проверка корректности выполняется слоями валидации VK_LAYER_KHRONOS_validation.

Результаты

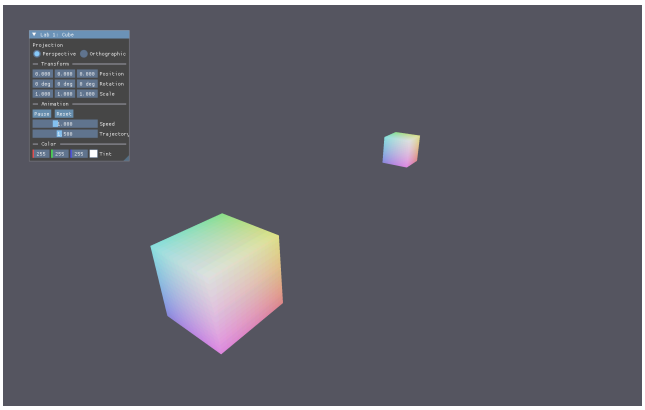


Рис 1. - Рабочее окно программы

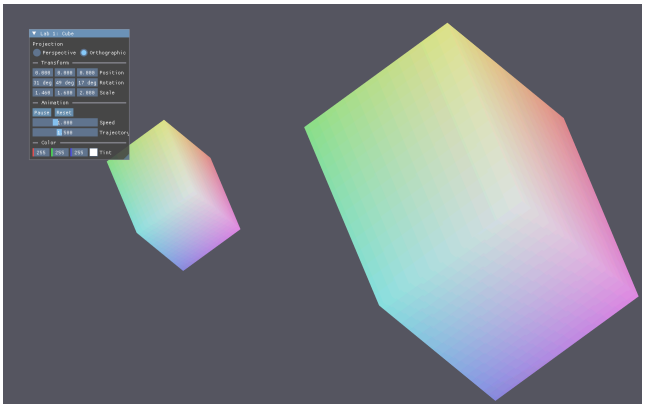


Рис 2. - ортографическая проекция и афинные преобразования

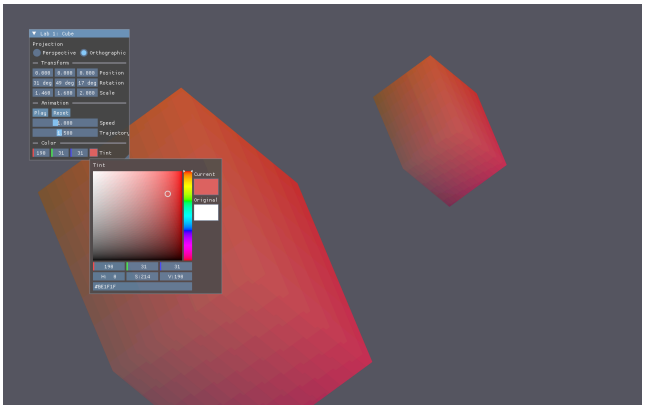


Рис 3. - Изменения цвета фигур

Выводы

В ходе лабораторной работы я изучил(а) основы 3D-графики на примере построения и отрисовки куба средствами Vulkan. На практике освоил конвейер рендеринга: создание вершинных и индексных буферов, графического пайплайна и дескрипторных наборов, запись и отправку командных буферов. Разобрался с математикой 3D-преобразований: модельная матрица (аффинные преобразования), матрица вида, перспективная и ортографическая проекции, особенности клип-пространства Vulkan. Научился передавать данные в шейдеры через uniform-буферы, реализовал интерактивное управление сценой через ImGui, анимацию по параметрически заданной траектории и отрисовку нескольких объектов с независимыми наборами дескрипторов. Получил опыт отладки Vulkan-приложения с помощью слоёв валидации.